# CUPS Printer Discovery Security Lab

## 1. Introduction

This project investigates insecure legacy CUPS printer discovery in an isolated Docker laboratory. The goal is to demonstrate how a crafted UDP/631 browse advertisement can cause vulnerable `cups-browsed` to process an attacker controlled printer endpoint, and to evaluate Scapy-based detection and an allowlist mitigation.

The authoritative assignment is [../assignment.md](../assignment.md). No public hosts, physical printers, or uncontrolled networks were used.

## 2. Vulnerability background

The lab uses the Vulhub `cups-browsed:2.0.1` environment associated with the 2024 CUPS printer-discovery vulnerability family. The relevant behavior is trust in unauthenticated legacy browse advertisements. A browse datagram can identify a remote host, port, queue, and display fields; the victim then tries to create or refresh a local queue and performs IPP requests.

This report demonstrates the discovery and connection behavior only. It does not include remote-code-execution payloads, malicious PPDs, or command injection.

## 3. System architecture and network

Docker Compose creates an internal-only `172.28.77.0/24` network:

| Component | Address | Process |
|---|---:|---|
| Ubuntu Scapy attacker | 172.28.77.10 | Python, Scapy, tcpdump |
| Vulnerable victim | 172.28.77.20 | cups-browsed 2.0.1 and CUPS |
| Virtual printer | 172.28.77.30 | OpenPrinting `ippserver` on TCP/631 |

The attacker has only `NET_RAW`, no published ports, and host-mounted evidence directories. The victim image uses `linux/amd64` emulation on Apple Silicon. The host can be macOS Docker Desktop; Kali may run separately in VMware Fusion.

Environment evidence is in [network-inspect.txt](../evidence/logs/network-inspect.txt),
[attacker-ip.txt](../evidence/logs/attacker-ip.txt), and
[compose-ps.txt](../evidence/logs/compose-ps.txt).

## 4. Attack-tool design

[`cups_attack.py`](../cups_attack.py) provides:

- `build_advertisement`: constructs IP/UDP/Raw packets with source and destination addresses, UDP/631, a five-field browse payload, UTF-8 fields, and safe quoting.
- `build_variation`: creates baseline, missing-URI, invalid-port, and malformed-field cases.
- `write_log`: records whether a packet was generated or transmitted, the addresses, variation, and payload.
- `main`: validates IPv4 addresses, port/range, repeat count, interval, laboratory destination, and explicit live authorization before writing a PCAP or calling Scapy `send`.

The UDP datagram itself is the protocol boundary. A trailing newline was tested and rejected by this cups-browsed build, so the implementation intentionally ends after the final quoted field.

## 5. Detection-tool design

[`cups_detector.py`](../cups_detector.py) provides:

- `parse_advertisement`: decodes quoted fields with `shlex`, checks the five expected fields, validates integer message fields, and parses the IPP URI and port.
- `inspect_packets`: maintains separate advertiser and printer allowlists, detects malformed fields, records advertised endpoints, correlates later TCP connections with those endpoints, and detects repeated signatures within a configurable time window.
- `alert`: emits timestamps, rule IDs, severity, source/destination, and a descriptive explanation.
- `write_results`: exports JSON or CSV.
- `main`: selects offline `rdpcap` or live `sniff`, validates CLI values, and handles empty captures and input errors explicitly.

Rule IDs are `DISCOVERY-001` through `DISCOVERY-004` and `IPP-001`.

## 6. Experiment methodology

Generated fixtures were used to test deterministic tool behavior. Independent real captures were then collected inside the Docker namespaces because host macOS `tcpdump` returned `Operation not permitted`. The capture filters were:

```text
udp port 631
udp port 631 or tcp port 631
```

`tcpdump -r` verified each capture. The attacker namespace captured crafted outgoing datagrams; the printer namespace captured the legitimate datagram and the victim's subsequent TCP/631 traffic.

## 7. Actual experiments and results

### Experiment 1: normal discovery and IPP communication

The valid legacy-compatible browse URI was:

```text
ipp://172.28.77.30:631/printers/lab
```

The real capture [legitimate-clean.pcap](../pcap_files/real/legitimate-clean.pcap)
contains the UDP advertisement followed by the TCP handshake and IPP request exchange. The victim log records:

```text
Remote host: 172.28.77.30; Port: 631; Remote queue name: lab
Found CUPS queue/class: lab on host 172.28.77.30.
Requested IPP attributes ... ipp://172.28.77.30:631/printers/lab
ERROR: Unable to create print queue, ignoring printer.
```

The virtual printer returned 7166 bytes to a direct IPP `Get-Printer-Attributes` request at its actual `/ipp/print` endpoint, but `ipptool` reported `client-error-bad-request` because the default ippserver advertises an invalid empty `uri-security-supported` keyword. This is recorded in [ippserver-direct-test.log](../evidence/logs/ippserver-direct-test.log).

Thus, discovery and communication succeeded, but queue creation did not; the report does not claim a successful print queue.

Advertising the standards-oriented URI `ipp://172.28.77.30:631/ipp/print` was also tested. The victim logged `Don't understand URI` before opening IPP. This is a measured compatibility limitation of the legacy browse parser, not an invented successful result.

### Experiments 2 and 3: unauthorized and manipulated discovery

| ID | Input | Real PCAP | Detector result |
|---|---|---|---|
| 2 | attacker `.10`, endpoint `.40:631`, `/printers/lab` | [unauthorized.pcap](../pcap_files/real/unauthorized.pcap) | 2 alerts: unauthorized advertiser and suspicious endpoint |
| 3a | same advertisement repeated 3 times | [repeated.pcap](../pcap_files/real/repeated.pcap) | 7 alerts including repeated discovery |
| 3b | advertised port `99999` | [invalid-port.pcap](../pcap_files/real/invalid-port.pcap) | malformed/invalid URI alert |
| 3c | `not-a-browse-message` | [malformed.pcap](../pcap_files/real/malformed.pcap) | malformed discovery alert |

Corresponding JSON results are under [evidence/detection_results](../evidence/detection_results/).

### Experiment 4: detection evaluation

The detector produced zero alerts for the allowlisted legitimate capture: [real-legitimate.json](../evidence/detection_results/real-legitimate.json). It detected all four real attack cases with the expected rule classes. The detector still reports the attack packet after mitigation, correctly measuring what was sent; the victim log measures whether the packet was accepted.

## 8. Detection evaluation

| Outcome | Measured evidence |
|---|---|
| True negative | Authorized real capture: zero detector alerts |
| True positive | Unauthorized, repeated, invalid, and malformed captures detected |
| False positive observed | None in the allowlisted legitimate capture |
| False negative observed | None for the tested real packet cases |

This is not a claim of complete protocol coverage. Encrypted traffic, fragmentation, alternate discovery formats, and endpoint changes outside the allowlists can be missed or require additional rules. Periodic advertisements are only repetition alerts when they exceed the configured threshold inside the configured time window.

## 9. Mitigation and before/after comparison

The reversible mitigation is:

```text
BrowseRemoteProtocols cups
BrowseOrder Allow,Deny
BrowseAllow 172.28.77.30
```

It is applied with:

```bash
```

```
docker compose -f docker-compose.yml -f docker-compose.mitigated.yml up -d victi
m
```

The authorized `.30` advertisement was accepted and caused IPP requests. The attacker `.10` advertisement was rejected with:

```text
browse packet from 172.28.77.10 disallowed
```

This is in [live-mitigated-victim.log](../evidence/logs/live-mitigated-victim.log).
The before and after unauthorized packets are [unauthorized.pcap](../pcap_files/real/unauthorized.pcap) and [mitigated-unauthorized.pcap](../pcap_files/real/mitigated-unauthorized.pcap). The detector still flags both packets because it evaluates traffic observed on the network; the mitigation changes victim acceptance and downstream IPP behavior, not the attacker's ability to emit a packet.

## 10. Architecture and execution walkthrough

### Attack execution

1. CLI arguments enter `cups_attack.main`.
2. Validators enforce IPv4, port, repeat, interval, fixed victim, and lab network constraints.
3. `build_variation` calls `build_advertisement`.
4. Scapy creates IP/UDP/Raw bytes with UDP source and destination port 631.
5. PCAP/log output is written. Only `--authorize-lab` permits `send`.
6. Docker routes the datagram over the internal bridge to `.20`.
7. cups-browsed parses the browse fields and may create a queue.
8. CUPS initiates TCP/631 IPP requests to the advertised endpoint.

### Detection execution

1. `rdpcap` or live `sniff` supplies packets to `inspect_packets`.
2. UDP/631 payloads pass through `parse_advertisement`.
3. Valid endpoints are stored for later connection correlation.
4. Allowlists, malformed fields, endpoint rules, repetition window, and correlated TCP rules create structured `alert` records.
5. `write_results` serializes JSON or CSV with rule ID and evidence fields.

### Mitigation execution

1. The Compose override mounts `cups-browsed.conf`.
2. cups-browsed loads `BrowseAllow 172.28.77.30`.
3. An advertisement from `.30` is accepted; `.10` is rejected before queue creation.
4. The victim log records the decision, and no unauthorized IPP request follows the rejected advertisement.

## 11. Code-reading walkthrough

Recommended function order:

1. [`cups_attack.py:validate_ip`](../cups_attack.py) and [`validate_port`](../cups_attack.py): input guards used by argparse.
2. [`build_advertisement`](../cups_attack.py): packet construction and field encoding; raises `ValueError` for unsafe fields or invalid direct calls.
3. [`build_variation`](../cups_attack.py): controlled experiment mutations.
4. [`main`](../cups_attack.py): output side effects and explicit live safety.

5. [`parse_advertisement`](../cups_detector.py): protocol decoding and URI validation.
6. [`inspect_packets`](../cups_detector.py): state tracking, detection rules, and correlation.
7. [`write_results`](../cups_detector.py): JSON/CSV output.
8. [`main`](../cups_detector.py): capture selection and error handling.

The tests in [tests/test_tools.py](../tests/test_tools.py) are a compact executable explanation of expected return values, errors, and alert behavior.

## 12. Verification

Commands executed:

```bash
python3 -m pytest -q
python3 -m py_compile cups_attack.py cups_detector.py
docker compose config
docker compose build attacker printer
docker compose up -d
tcpdump -r pcap_files/real/legitimate-clean.pcap -nn
python3 cups_detector.py --pcap pcap_files/real/unauthorized.pcap ...
docker compose -f docker-compose.yml -f docker-compose.mitigated.yml up -d victi
m
```

Final automated result: **11 passed**. All real PCAPs are readable, JSON and CSV exports were generated, Docker Compose validated, the attacker address was verified as `172.28.77.10`, and the mitigation was observed in victim logs.

## 13. Limitations

- GUI screenshots were not fabricated; text evidence and precise screenshot instructions are in [evidence/screenshots/README.txt](../evidence/screenshots/README.txt).
- Host-level Docker bridge capture is unavailable without elevated macOS permissions; namespace captures are the documented workaround.
- `ippserver` responds at `/ipp/print`, while this cups-browsed legacy parser only processes the `/printers/<queue>` browse form. Its response also contains an invalid empty `uri-security-supported` attribute, so queue creation is not claimed as successful.
- The detector is not a full IPP implementation.

## 14. Ethical considerations

Testing is restricted to the private Docker network and authorized containers. The live attack requires an explicit safety flag and fixed lab destination. No public targets, physical printers, exploit payloads, or persistence mechanisms are used.

## 15. Conclusion

The lab demonstrates that vulnerable cups-browsed accepts a legacy printer advertisement from an unauthorized source and attempts IPP communication with the advertised endpoint. The detector identifies the tested attack classes, and the allowlist mitigation accepts `.30` while rejecting `.10`. The evidence also shows why discovery acceptance, IPP communication, and successful queue creation must be reported as separate outcomes.

## 16. References

- Assignment specification: [../assignment.md](../assignment.md)

- Vulhub CUPS laboratory: https://github.com/vulhub/vulhub/tree/master/cups-browsed/CVE-2024-47177
- CUPS browsing configuration documentation: https://manpages.debian.org/bookworm/cups-browsed/cups-browsed.conf.5.en.html
- Scapy documentation: https://scapy.readthedocs.io/